

Sobre: Model Context Protocol (MCP) Tool Descriptions Are Smelly! Towards Improving AI Agent Efficiency with Augmented MCP Tool Descriptions

Total de 26 Refências:

- Sendo 3 De artigos já publicados
 - **EduMSRA: A Multi-Source Educational Research Agent Integrating Retrieval-Augmented Generation and Model Context Protocol for Adaptive Intelligent Tutoring Systems**
 - publicado na MDPI
 - no link <https://www.mdpi.com/2076-3417/16/9/4400>
 - Momento da citação:

Security analyses have identified critical concerns for educational deployment. Zhang et al. [26] established a 12-category attack taxonomy including name-collision, prompt injection, and data exfiltration across 2000 attack instances. Radosevich and Hal-loran [27] demonstrated malicious code execution and credential theft via MCP servers. Hasan et al. [49] found that 97.1% of MCP tool descriptions contain at least one “smell”, with 56% failing to state purpose clearly, underscoring the importance of carefully crafted educational tool descriptions. Protocol comparison studies [50,51] have positioned MCP within the broader landscape of agent interoperability protocols (A2A, ACP, ANP), recommending MCP as the starting point for enterprise adoption.
 - **An LLM-agent interface for end-to-end probabilistic seismic hazard and risk analysis**
 - publicado na Nature
 - no link <https://www.nature.com/articles/s44304-026-00262-z>
 - Sem referencia direta
 - **Lean Service Catalogs for Agentic Systems: A Study of Schema Compression and Caching Economics**
 - publicado na IEEE
 - no link https://ieeexplore.ieee.org/abstract/document/11670139/?casa_token=4PqBNYdCYeEAAAAA:IKb0j0zeXmY67jNAchsR1PKHFMPTuBL1iTvXM5q-DQIn4m6a_VsC8e_xvFGycPL5tKcAHaAR1m0
 - Momento da citação:

Stein [1] surveyed 177,436 MCP tools: 67% are in software development; the share of state-modifying “action” tools rose from 27% to 65% in 15 months. Fan et al. curated MCP-ToolBench++ from 4,000+ servers [6]. Hasan et al. found that

— — — — — [... continua na página seguinte ...] — — — — —

compact tool descriptions often match the behavior of verbose ones [7].

Description Smell

- O termo também aparece no trabalho

From Docs to Descriptions: Smell-Aware Evaluation of MCP Server Descriptions

PEIRAN WANG*, UCLA, USA

YING LI*, UCLA, USA

YUQIANG SUN, NTU, Singapore

CHENGWEI LIU, NTU, Singapore

YANG LIU, NTU, Singapore

YUAN TIAN, UCLA, USA

The Model Context Protocol (MCP) has rapidly become a de facto standard for connecting LLM-based agents with external tools via reusable MCP servers. In practice, however, server selection and onboarding rely heavily on free-text tool descriptions that are intentionally loosely constrained. Although this flexibility largely ensures the scalability of MCP servers, it also creates a reliability gap that descriptions often misrepresent or omit key semantics, increasing trial-and-error integration, degrading agent behavior, and potentially introducing security risks. To this end, we present the first systematic study of description smells in MCP tool descriptions and their impact on usability. Specifically, we synthesize software/API documentation practices and agentic tool-use requirements into a four-dimensional quality standard—accuracy, functionality, information completeness, and conciseness, covering 18 specific smell categories. Using this standard, we conducted a large-scale empirical study on a well-constructed dataset of 10,831 MCP servers. We find that description smells are pervasive (e.g., 73% repeated tool names; thousands with incorrect parameter semantics or missing return descriptions), reflecting a “code-first, description-last” pattern. Through a controlled mutation-based study, we show these smells significantly affect LLM tool selection, with functionality and accuracy having the largest effects (+11.6% and +8.8%, $p < 0.001$). In competitive settings with functionally equivalent servers, standard-compliant descriptions reach 72% selection probability (260% over a 20% baseline), demonstrating that smell-guided remediation yields substantial practical benefits. We release our labeled dataset and standards to support future work on reliable and secure MCP ecosystems.

Hassan



Ahmed E. Hassan

SEGUIR

Mustafa Prize Laureate, ACM/IEEE/NSERC Steacie Fellow, ACM Influential/IEEE Distinguished Educator
E-mail confirmado em cs.queensu.ca - [Página inicial](#)

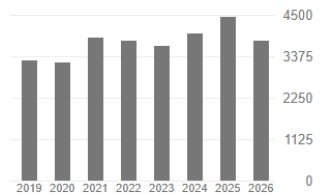
[Mining Software Repositories](#) [Software Analytics](#) [Empirical Software Engineering](#) [Software](#)

TÍTULO	CITADO POR	ANO
Predicting faults using the complexity of code changes AE Hassan 2009 IEEE 31st international conference on software engineering, 78-88	1002	2009
A large-scale empirical study of just-in-time quality assurance Y Kamei, E Shihab, B Adams, AE Hassan, A Mockus, A Sinha, ... IEEE Transactions on Software Engineering 39 (6), 757-773	994	2012
Studying the effectiveness of application performance management (apm) tools for detecting performance regressions for web applications: An experience report TM Ahmed, CP Bezemer, TH Chen, AE Hassan, W Shang Proceedings of the 13th International Conference on Mining Software ...	902	2016
What are developers talking about? an analysis of topics and trends in stack overflow A Barua, SW Thomas, AE Hassan Empirical software engineering 19 (3), 619-654	885	2014
An empirical comparison of model validation techniques for defect prediction models C Tantithamthavorn, S McIntosh, AE Hassan, K Matsumoto IEEE Transactions on Software Engineering 43 (1), 1-18	757	2016
What do mobile app users complain about? H Khalid, E Shihab, M Nagappan, AE Hassan IEEE software 32 (3), 70-77	687	2014
Revisiting the impact of classification techniques on the performance of defect prediction models B Ghotra, S McIntosh, AE Hassan 2015 IEEE/ACM 37th IEEE International Conference on Software Engineering 1 ...	562	2015
The impact of automated parameter optimization on defect prediction models C Tantithamthavorn, S McIntosh, AE Hassan, K Matsumoto IEEE Transactions on Software Engineering 45 (7), 683-711	549	2018
An empirical study of the impact of modern code review practices on software quality S McIntosh, Y Kamei, B Adams, AE Hassan Empirical Software Engineering 24 (6), 2445-2468	549	2016

OBTER MEU PRÓPRIO PERFIL

Citado por [VER TODOS](#)

	Todos	Desde 2021
Citações	44151	23684
Índice h	112	80
Índice i10	466	363



Acesso público [VER TODOS](#)

13 artigos [99 artigos](#)
não disponível [disponível](#)

Com base nas autorizações de financiamento

Coautores [VER TODOS](#)

	Bram Adams Queen's University	>
	Zhen Ming (Jack) Jiang York University, Toronto, Canada	>
	Shane McIntosh Queen's University	>

Possível substituto

- nome: ToolScope: Enhancing LLM Agent Tool Use through Tool Merging and Context-Aware Filtering
- link: https://aclanthology.org/2026.acl-long.1573/?utm_source=chatgpt.com
- ano: 2026
- publicado em: ACL Anthology
- rubrica

You are an expert evaluator assessing the quality of tool documentation.

Given a tool's name, function signature, and docstring, decide how clear and helpful the documentation is for someone who needs to call the tool correctly.

Judge the documentation on three evidence pillars:

P Parameters

- Do parameter names and counts match the signature?
- Are required parameters described accurately?
- Are optional parameters or default values acknowledged?

I Inputs & Constraints

- Are input types, roles, and typical values clear?
- Are important constraints, ranges, or usage conditions provided?

B Behavior & Purpose

- Is it clear—at least at a high level—what the tool does when invoked?
- Does the description indicate what the caller can expect in return (or is it self-evident)?

Scoring rubric (1–5):

Score 5 – Excellent

Parameters (P): All required and optional parameters are fully documented; naming matches the signature; defaults are stated when non-trivial.

Inputs & Constraints (I): Types plus examples or constraints are provided for almost every field.

Behavior & Purpose (B): Clear, precise description of functionality and return/output.

Overall Quality: Ready for production use.

Score 4 – Good

Parameters (P): Required parameters are fully documented; most optional parameters are mentioned, with only minor detail gaps.

Inputs & Constraints (I): Types are present, but a few examples or constraints are missing.

Behavior & Purpose (B): Behavior is described clearly, but return/output is only briefly mentioned or assumed obvious.

Overall Quality: Very usable; small refinements desirable.

Score 3 – Fair

Parameters (P): Required parameters are documented and correct, but several optional parameters are missing or vague.

Inputs & Constraints (I): Basic types or roles are stated, but little constraint information is given.

Behavior & Purpose (B): Behavior is conveyed in one or two somewhat vague sentences but is still understandable.

Overall Quality: Usable with modest guesswork.

Score 2 – Poor

Parameters (P): Some required parameters are undocumented or mismatched with the signature.

Inputs & Constraints (I): Types are unclear or absent; no constraint details are given.

Behavior & Purpose (B): Behavior is barely described, and purpose is hard to infer.

Overall Quality: High risk of misuse.

Score 1 – Very Poor

Parameters (P): Few or no parameters are documented, or documentation is misleading.

Inputs & Constraints (I): Input expectations are absent or incorrect.

Behavior & Purpose (B): The tool's purpose is not conveyed.

Overall Quality: Documentation offers virtually no guidance.

Respond with exactly two lines:

1. **Score** (integer 1–5)

2. **One-sentence justification**

Now, evaluate the following tool:

Tool name: {tool_name}

Tool signature: {tool_signature}

Tool description: {tool_description}